

SmartStudy Cloud Agent

Implementation Summary

Esiev Kerim Houdaibi Salman Sewif Mohamed Mabrouk Bouzouita Chahine

INFO-H505 Cloud Computing – Prof. Dimitris SACHARIDIS – Academic Year 2025–2026

1 Project Context

SmartStudy Cloud Agent was developed for the course **INFO-H505 Cloud Computing**, taught by **Dimitris SACHARIDIS** during the academic year **2025–2026**. The project objective was to build a production-oriented cloud application on Google Cloud Platform. The application had to allow a user to upload lecture PDFs to a cloud folder and then interact with an AI tutor able to answer questions from those uploaded documents.

The project follows a **Retrieval-Augmented Generation** architecture. In a standard chatbot, the language model may answer from its general training data. In SmartStudy, the model first receives document context retrieved from the uploaded course material. This makes the answer more reliable, because the response is grounded in the student’s documents and can cite the source chunks used. This design directly supports the academic goal of the project: the assistant should behave like a tutor for exam preparation, not like a generic search engine.

The final implementation is mainly based on two files. The first file, `main.py`, contains the automated ingestion pipeline deployed as a Google Cloud Function. The second file, `app.py`, contains the Streamlit interface and the RAG orchestration used to answer student questions. Together, these files implement the required features: GCS upload, Cloud Function trigger, PDF extraction, LangChain chunking, Vertex AI embeddings, MongoDB Atlas Vector Search, Gemini 2.5 Flash generation, and a formal academic tutor persona.

2 Final Cloud Architecture

The system is event-driven. A PDF upload to Google Cloud Storage creates an object-finalized event. This event is captured by Eventarc and triggers the Cloud Function Gen2 service. The function processes the PDF and stores vectorized chunks in MongoDB Atlas. The Streamlit application then retrieves the most relevant chunks when the student asks a question.

The environment flowchart can be found in the same folder.

The selected advanced feature is the **Streamlit web interface**. This gives the project a usable front-end instead of limiting it to terminal-based testing. It also makes the live demonstration clearer, because a student can interact with the tutor through a chat interface.

3 Automated Ingestion Pipeline

The ingestion pipeline is implemented in `main.py`. The file defines a Cloud Function called `ingest_pdf`. When an object is finalized in the Google Cloud Storage bucket, the function receives the event metadata: bucket name, object name, and content type. The function first checks that the uploaded object is a PDF. If the file is not a PDF, it is skipped. This prevents unrelated files from being inserted into the vector database.

For valid PDFs, the function downloads the document from GCS into the temporary `/tmp` directory of the Cloud Function runtime. It then extracts the text with `pypdf`. If no extractable text is found, the function stops cleanly. This is important because image-only or scanned PDFs may not contain directly readable text.

The extracted text is split using LangChain’s `RecursiveCharacterTextSplitter`. The chosen configuration uses chunks of 1000 characters with an overlap of 200 characters. This keeps enough context inside each chunk while reducing the risk of losing information at chunk boundaries. The chunks are then embedded with Vertex AI `text-embedding-005`. Embeddings are generated in batches of five chunks, which makes the function more stable than sending all chunks at once.

Each chunk is stored in MongoDB Atlas with its text, embedding vector, source file, chunk index, bucket name, and object name. The implementation also creates a stable identifier from the source path and the chunk number. MongoDB receives `UpdateOne` operations with `upsert=True`, so reprocessing the same file updates existing chunks instead of creating duplicates. This makes the ingestion step closer to a production pipeline than a simple manual script.

4 MongoDB Atlas and Retrieval

MongoDB Atlas is used as the vector database. A configured Atlas collection stores the document chunks and their embeddings. A vector index is defined on the embedding field. When a question is asked, the question is also converted into an embedding. MongoDB Atlas Vector Search compares this query vector with the stored chunk vectors and returns the most relevant passages.

This retrieval strategy is semantic rather than purely lexical. The system does not only search for identical keywords; it searches for meaning. This is important for a tutor application because a student may ask a question using different wording from the course PDF. The retrieved metadata also allows the final answer to mention where the information came from, using the source file and the chunk number.

The MongoDB layer is intentionally treated as a configurable component rather than as a fixed part of the Python logic. Database names, collection names, index names, connection strings, and retrieval limits are deployment parameters. In practice, the code reads these values from its execution environment, while the actual Atlas cluster and vector index are configured outside the application code. This separation is important because the same ingestion and retrieval logic can be reused with another database, collection, or index simply by changing the deployment configuration, without rewriting the core pipeline.

5 Tutor Application and RAG Orchestration

The tutor interface is implemented in `app.py`. It uses Streamlit to provide a chat-like web interface. When the student asks a question, the application embeds the question with Vertex AI and performs a `$vectorSearch` aggregation in MongoDB Atlas. The retrieved chunks are then formatted into a context block.

LangChain is used to organize the generation step. The application uses `ChatPromptTemplate` to combine the system instruction, the retrieved context, and the student's question. It calls Gemini 2.5 Flash through `ChatVertexAI`, and the output is parsed with `StrOutputParser`. In practice, the RAG chain is:

Retrieved context + question → LangChain prompt → Gemini 2.5 Flash → Tutor answer

The application includes basic robustness improvements. MongoDB errors are caught with `PyMongoError`, the database client is closed after retrieval, and generation errors are displayed to the user instead of crashing the interface. Streamlit session state is also used to keep the chat history visible during the interaction.

6 Tutor Persona

The tutor behavior is controlled by a strict system instruction. SmartStudy is defined as a formal academic tutor. It must answer only from the provided context passages, avoid speculation, cite the source chunks, and clearly state when the uploaded course material is insufficient. The answer format is also structured: an explanation, a list of sources used, and one comprehension check question.

This persona is essential to the project. The objective is not only to generate fluent text, but to produce grounded academic assistance. The tutor should help students understand their course material and verify their comprehension. The final follow-up question makes the interaction more pedagogical, because it pushes the student to actively reflect on the answer.

7 From Sandbox to Live Cloud

The first phase of the project was completed in the Google Skills sandbox environment. This phase was used to validate the technical workflow without using the final live cloud credits. In the sandbox, the team confirmed the complete ingestion chain: a PDF upload could trigger a Cloud Function, the function could download the file, extract text, create LangChain chunks, generate Vertex AI embeddings, and insert the resulting vectors into MongoDB Atlas.

Before deploying the full ingestion logic, the team first tested a minimal Cloud Function. This was useful to verify that the infrastructure itself worked: Google Cloud Storage produced an object-finalized event, Eventarc routed this event correctly, and the Cloud Function executed and produced logs. Once this trigger was validated, the simple test function was replaced by the real PDF ingestion function.

The sandbox was useful, but temporary and constrained. Some regions were restricted, so `us-central1` was used and kept for consistency in the live project. This avoided changing the bucket location, function region, and Vertex AI location during the migration. The live deployment then moved the project into the real Google Cloud environment. The main technical change was the move from a lab-style workflow to a permanent cloud-native pipeline. In the lab, ingestion could be tested step by step. In the final system, Google Cloud Storage became the official entry point, and every PDF upload automatically launches the processing pipeline.

The live environment also required additional configuration: creating the real GCS bucket, enabling the required Google Cloud APIs, configuring Eventarc, setting IAM permissions, connecting the Cloud Function to MongoDB Atlas, and using Secret Manager for the MongoDB Atlas URI. This matters because Cloud Functions Gen2 depends on several cloud services around it, including Cloud Run, Cloud Build, Artifact Registry, Pub/Sub, Logging, Vertex AI, and Secret Manager.

8 Validation and Limitations

The ingestion pipeline was validated in the live Google Cloud project. A test PDF triggered the Cloud Function successfully, and the logs confirmed the full sequence: download, text extraction, chunking, embedding generation, and MongoDB upsert. A real course-related PDF was also uploaded and processed into multiple chunks. The RAG behavior was then tested by asking questions about uploaded documents. For relevant questions, the system retrieved sources and generated contextualized answers. For unrelated questions, it correctly indicated that the uploaded material did not contain enough information, which shows that the grounding instruction reduces hallucinations.

The current implementation still has limitations. Text extraction depends on `pypdf`, so scanned documents and diagrams are not deeply analyzed. The metadata could also be improved by storing page numbers, upload timestamps, document titles, and user identifiers. Finally, if many documents are stored in the same MongoDB collection, the interface should ideally let the student select a specific PDF and pass that selection as a retrieval filter.

9 Conclusion

SmartStudy satisfies the main requirements of the INFO-H505 project. It provides automated PDF ingestion through Google Cloud Storage and Cloud Functions, document chunking through LangChain, embeddings through Vertex AI, vector storage and retrieval through MongoDB Atlas, answer generation through Gemini 2.5 Flash, and a formal tutor persona designed for academic use.

The most important evolution from the lab to the live cloud environment was automation. The sandbox validated the architecture, while the final implementation transformed it into a persistent event-driven cloud application. The Streamlit interface completes the project by providing a practical way for users to interact with the tutor during the final demonstration.